

19. Ejercicio de videojuego

Pet, Niebla y Wordly

Jonathan Ricardo Serna Galina
20074902

Programación Asistida

1. Descripción general de la carpeta 19

La carpeta 19vo Ejercicio videojuego contiene Castillo Matemático, un juego educativo desarrollado con Python y Streamlit. El jugador recorre un castillo y derrota enemigos al resolver operaciones aritméticas dentro de un límite de tiempo. Cada acierto permite avanzar y ganar oro; una respuesta incorrecta o el tiempo agotado reduce la vida. La dificultad aumenta conforme progresa la partida.

La configuración de main define nueve habitaciones, 100 puntos de vida y cero monedas iniciales. Las tres ramas documentadas añaden una mascota virtual, un evento de niebla y un comodín de palabras, respectivamente.

2. Ejercicio Pet

2.1 Descripción

Pet incorpora una mascota con tres indicadores: alimentación, energía y felicidad. Todos comienzan en 100 puntos y disminuyen 10 puntos al superar cada habitación. El jugador puede recuperar estos valores mediante las acciones Alimentar, Jugar y Dormir.

En cada habitación se elige al azar una oferta por acción. Cada producto tiene un precio en oro y una cantidad de puntos de recuperación. Una oferta pendiente se puede comprar una sola vez si hay monedas suficientes, o se puede omitir. Las compras descuentan el oro y recuperan el indicador correspondiente hasta un máximo de 100 puntos.

La rama integra estos datos en el estado del juego y muestra los indicadores y las ofertas en la interfaz. El siguiente listado reproduce completo el archivo pet.py.

2.2 Código: pet.py

Rama: ecojrsg/opencode-mascota-19vo

```
1 import random
2
3
4 METER_MAX = 100
5 ROOM_DECAY = 10
6
7 ACTIONS = {
8     "alimentar": {"label": "🍲 Alimentar", "stat": "well_fed"},
9     "jugar": {"label": "🎮 Jugar", "stat": "happiness"},
10    "dormir": {"label": "😴 Dormir", "stat": "energy"},
11 }
12
13 OFFER_CATALOG = {
14     "alimentar": [
15         {"name": "Croquetas", "price": 8, "restore": 25},
16         {"name": "Comida casera", "price": 16, "restore": 45},
17         {"name": "Salmón", "price": 24, "restore": 60},
18     ],
19     "jugar": [
20         {"name": "Pelota", "price": 7, "restore": 20},
21         {"name": "Cuerda", "price": 12, "restore": 35},
22         {"name": "Puzzle", "price": 18, "restore": 50},
23     ],
24     "dormir": [
25         {"name": "Cojín", "price": 8, "restore": 25},
26         {"name": "Cama cómoda", "price": 15, "restore": 45},
27         {"name": "Manta térmica", "price": 22, "restore": 60},
28     ],
29 }
30
31
32 def new_pet() -> dict:
33     return {stat: METER_MAX for stat in (action["stat"] for action in ACTIONS.values())}
34
35
36 def new_room_offers() -> dict:
37     return {
38         action: {"**random.choice(options), "status": "pending"}
39         for action, options in OFFER_CATALOG.items()
40     }
41
42
43 def purchase_offer(state: dict, action: str) -> bool:
44     offer = state["offers"][action]
45     if offer["status"] != "pending" or state["gold"] < offer["price"]:
46         return False
47
48     state["gold"] -= offer["price"]
49     stat = ACTIONS[action]["stat"]
50     state["pet"][stat] = min(METER_MAX, state["pet"][stat] + offer["restore"])
51     offer["status"] = "purchased"
52     return True
53
54
55 def skip_offer(state: dict, action: str) -> bool:
56     offer = state["offers"][action]
57     if offer["status"] != "pending":
58         return False
59     offer["status"] = "skipped"
60     return True
61
62
63 def decay_pet(pet: dict) -> None:
64     for action in ACTIONS.values():
65         stat = action["stat"]
66         pet[stat] = max(0, pet[stat] - ROOM_DECAY)
```

2.3 Evidencia visual y repositorio

❤️ Vida

100

🪙 Oro

6

🗺️ Mapa

✅ → ✅ → 🏠 3 → 🏠 4 → 🏠 5 → 🏠 6 → 🏠 7 → 🏠 8 → 🏠 9

🐾 Tu mascota

🍽️ Bien alimentada: 100/100

⚡ Energía: 80/100

😊 Felicidad: 80/100

Ofertas de esta habitación

🍏 Alimentar

🎮 Jugar

🛌 Dormir

Salmón · 60 puntos · 24 🪙

Pelota · 20 puntos · 7 🪙

Cojín · 25 puntos · 8 🪙

Comprada

Omitida

Comprar · 8 🪙

Omitir

Figura 1. Partida de Pet después de superar dos habitaciones. Se observan los tres indicadores, una oferta comprada, otra omitida y el oro disponible. La alimentación se recuperó a 100; energía y felicidad permanecen en 80.

Liga del código: [Abrir pet.py de la rama Pet en GitHub](#)

3. Ejercicio Niebla

3.1 Descripción

Niebla añade un evento aleatorio que puede aparecer al entrar en una habitación. La posibilidad de ofrecer el evento es de una entre tres. El resultado de esa comprobación se conserva en el estado de la partida, por lo que actualizar la pantalla no vuelve a sortear la misma sala.

Cuando aparece el evento, el jugador inicia una ruleta con dos resultados posibles: Pase libre o Enemigo. La animación dura 2.4 segundos. Mientras la ruleta está ofrecida o girando, el temporizador de combate permanece en pausa.

Pase libre avanza a la siguiente habitación sin otorgar oro de combate. Enemigo conserva el encuentro normal y permite resolver la operación matemática. Al finalizar la ruleta se reanuda el tiempo del turno. La interfaz muestra el evento, la animación y el resultado; las reglas de la ruleta se encuentran en fog.py, reproducido completo a continuación.

3.2 Código: fog.py

Rama: ecojrsg/opencode-niebla-19vo

```
1  from __future__ import annotations
2
3  import random
4  import time
5
6  SPIN_SECONDS = 2.4
7  PASS = "Pase libre"
8  ENEMY = "Enemigo"
9
10
11 def check_room_event(state: dict, room: int, rng=None) -> dict:
12     """Roll once per room and keep the result in the game's session state."""
13     events = state.setdefault("fog_events", {})
14     key = str(room)
15     if key not in events:
16         rng = random if rng is None else rng
17         events[key] = {"status": "offered" if rng.randrange(3) == 0 else "none"}
18     return events[key]
19
20
21 def start_roulette(state: dict, room: int, rng=None, now: float | None = None) -> bool:
22     event = state.setdefault("fog_events", {}).get(str(room))
23     if not event or event["status"] != "offered":
24         return False
25
26     rng = random if rng is None else rng
27     event.update(
28         status="spinning",
29         result=rng.choice((PASS, ENEMY)),
30         started_at=time.time() if now is None else now,
31     )
32     return True
33
34
35 def resolve_roulette(
36     state: dict,
37     room: int,
38     settings: dict,
39     generator,
40     now: float | None = None,
41 ) -> str | None:
42     event = state.setdefault("fog_events", {}).get(str(room))
43     if not event or event["status"] != "spinning":
44         return None
45
46     now = time.time() if now is None else now
47     if now - event["started_at"] < SPIN_SECONDS:
48         return None
49
50     result = event["result"]
51     event["status"] = "passed" if result == PASS else "enemy"
52     event["resolved_at"] = now
53     state["started_at"] = now
54
55     if result == PASS:
56         state["fog_notice"] = {"room": room, "result": PASS}
57         state["room"] = room + 1
58         state["finished"] = state["room"] > settings["rooms"]
59         if not state["finished"]:
60             state["exercise"] = generator.generate(state["room"])
61
62     return result
```

3.3 Evidencia visual y repositorio

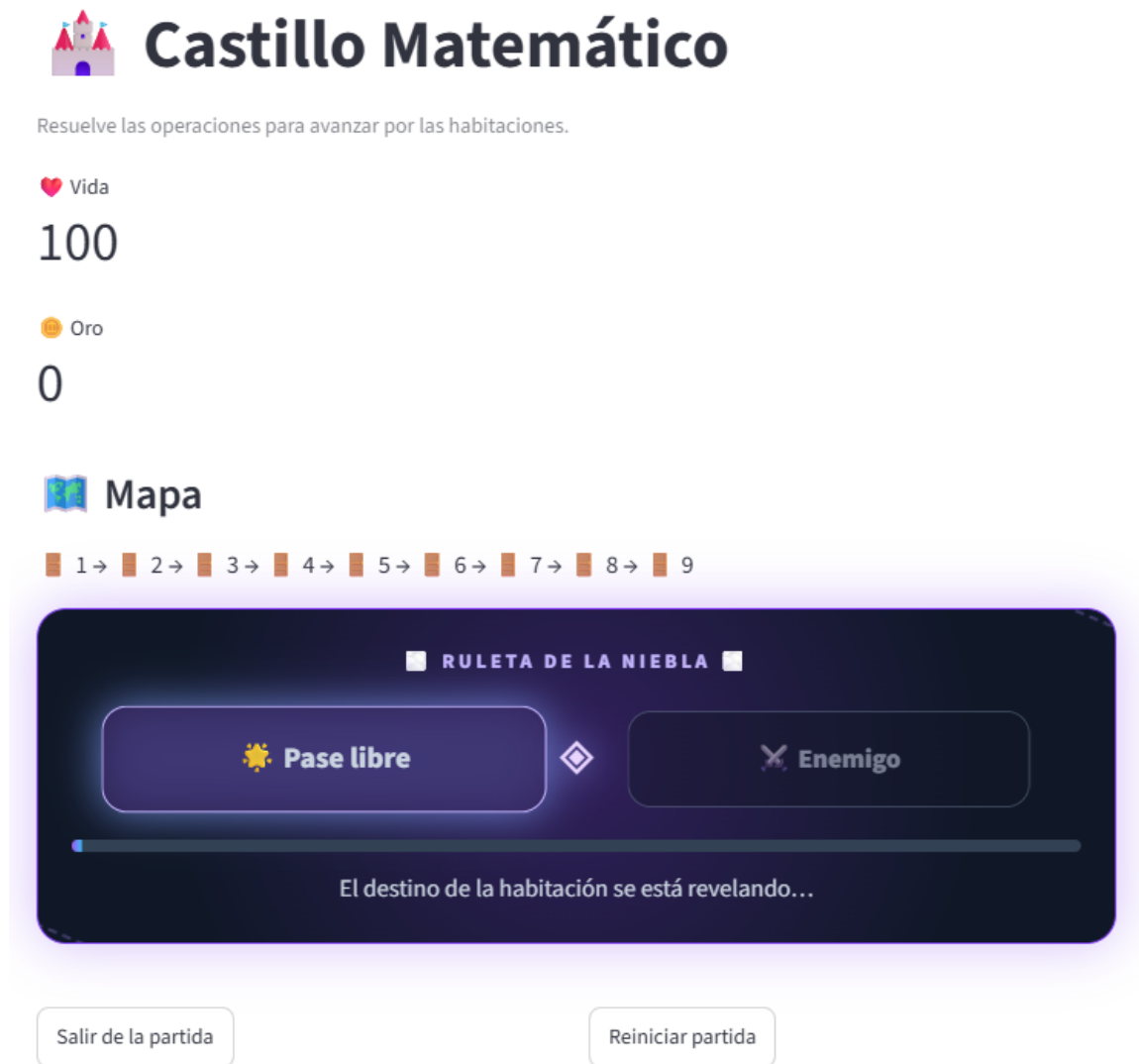


Figura 2. Ruleta de Niebla en funcionamiento en la habitación 1. Se muestran las alternativas Pase libre y Enemigo y el progreso de la animación. En esta ejecución el resultado posterior fue Enemigo.

Liga del código: [Abrir fog.py de la rama Niebla en GitHub](#)

4. Ejercicio Wordly

4.1 Descripción

Wordly añade un comodín de palabras a la aventura. En el repositorio la variante se denomina Wordle y está implementada en `wordle.py`. El jugador puede abrir este minijuego durante una partida activa; el temporizador de combate queda en pausa hasta cancelarlo o regresar al castillo.

El minijuego elige una palabra española de entre cinco y diez letras y permite hasta seis intentos. Las propuestas deben pertenecer a la lista disponible y tener la misma longitud que la palabra objetivo. La comparación ignora mayúsculas y tildes y conserva la ñ como una letra distinta.

Cada intento recibe pistas por letra: verde indica una coincidencia en la posición correcta, amarillo indica una letra presente en otra posición y gris indica una letra ausente. Las coincidencias exactas se cuentan primero para manejar correctamente las letras repetidas. El estado registra los intentos y determina si la partida continúa, se gana o se pierde.

La rama incorpora el tablero, la entrada de palabras y los controles para volver al castillo. El siguiente listado reproduce completo `wordle.py`.

4.2 Código: wordle.py (1/2)

Rama: ecojrsg/opencode-wordle-19vo

```
1  # @Last Modified by:   Jonathan Serna
2  # @Last Modified time: 2026-10-02 18:40:22
3
4  """Rules and vocabulary for the Wordle joker."""
5
6  from __future__ import annotations
7
8  import random
9  import unicodedata
10
11  MIN_WORD_LENGTH = 5
12  MAX_WORD_LENGTH = 10
13  MAX_ATTEMPTS = 6
14
15  SPANISH_WORDS = (
16      "amigo", "arbol", "avion", "barco", "brisa", "campo", "canto", "cielo",
17      "claro", "clase", "coche", "color", "dulce", "feliz", "fruta", "hogar",
18      "joven", "libro", "llave", "madre", "mundo", "noche", "papel", "pared",
19      "perro", "playa", "queso", "reloj", "sabor", "sueño", "tecla", "trigo",
20      "valor", "verde", "viaje", "vuelo", "animal", "bosque", "buscar", "cabeza",
21      "camino", "camisa", "ciudad", "comida", "cuerpo", "dinero", "fuerte", "jardin",
22      "lengua", "madera", "musica", "pelota", "piedra", "puente", "rapido", "sombra",
23      "sonido", "tomate", "verdad", "alegria", "caballo", "castigo", "colores", "cultura",
24      "cuidado", "familia", "hormiga", "lectura", "montaña", "naranja", "pintura", "secreto",
25      "sonrisa", "tarjeta", "trabajo", "ventana", "aventura", "castillo", "estrella", "juguetes",
26      "mariposa", "montañas", "sombrero", "tesorera", "valiosos", "viajeros", "volcanes", "caminante",
27      "carretera", "esperanza", "escaleras", "girasoles", "maravilla", "marineros", "personaje",
28      > "primavera",
29      "serpiente", "biblioteca", "calendario", "compañeros", "despedidas", "dinosaurio", "estudiante",
30      > "explorador",
31      "imaginario", "naturaleza", "revolucion", "tecnologia",
32  )
33
34  _WORD_SET = frozenset(SPANISH_WORDS)
35
36  CORRECT = "correct"
37  PRESENT = "present"
38  ABSENT = "absent"
39
40  def normalize_word(word: str) -> str:
41      """Ignore case and written accents while keeping ñ as a distinct letter."""
42      decomposed = unicodedata.normalize("NFD", word.strip().casefold())
43      letters: list[str] = []
44      for character in decomposed:
45          if unicodedata.combining(character):
46              if character == "\u0303" and letters and letters[-1] == "n":
47                  letters[-1] = "ñ"
48              continue
49          letters.append(character)
50      return "".join(letters)
51
52  def _known_word(word: str) -> str:
53      normalized = normalize_word(word)
54      if not normalized or any(
55          not ("a" <= letter <= "z" or letter == "ñ") for letter in normalized
56      ):
57          raise ValueError(
58              "La palabra solo puede contener letras del español."
59          )
60      if not MIN_WORD_LENGTH <= len(normalized) <= MAX_WORD_LENGTH:
61          raise ValueError("La palabra debe tener entre 5 y 10 letras.")
62      if normalized not in _WORD_SET:
63          raise ValueError("La palabra no está en la lista disponible.")
64      return normalized
65
```

4.2 Código: wordle.py (2/2)

Rama: ecojrsg/opencode-wordle-19vo

```
66
67 def new_game(target: str | None = None) -> dict:
68     """Create an isolated game, optionally with a deterministic target."""
69     answer = _known_word(
70         target if target is not None else random.choice(SPANISH_WORDS)
71     )
72     return {"target": answer, "guesses": [], "status": "playing"}
73
74
75 def evaluate_guess(target: str, guess: str) -> list[str]:
76     """Return Wordle colors, counting exact matches before misplaced letters."""
77     target = normalize_word(target)
78     guess = normalize_word(guess)
79     if len(target) != len(guess):
80         raise ValueError(
81             "La palabra debe tener la misma cantidad de letras que el objetivo."
82         )
83
84     feedback = [ABSENT] * len(guess)
85     remaining: dict[str, int] = {}
86     for index, (letter, answer_letter) in enumerate(zip(guess, target)):
87         if letter == answer_letter:
88             feedback[index] = CORRECT
89         else:
90             remaining[answer_letter] = remaining.get(answer_letter, 0) + 1
91
92     for index, letter in enumerate(guess):
93         if feedback[index] == ABSENT and remaining.get(letter, 0):
94             feedback[index] = PRESENT
95             remaining[letter] -= 1
96     return feedback
97
98
99 def submit_guess(game: dict, guess: str) -> list[str]:
100     """Validate and record a guess, updating the game outcome in place."""
101     if game["status"] != "playing":
102         raise ValueError("Esta partida ya terminó.")
103
104     normalized = _known_word(guess)
105     if len(normalized) != len(game["target"]):
106         raise ValueError(f"La palabra debe tener {len(game['target'])} letras.")
107
108     feedback = evaluate_guess(game["target"], normalized)
109     game["guesses"].append({"word": normalized, "feedback": feedback})
110     if normalized == game["target"]:
111         game["status"] = "won"
112     elif len(game["guesses"]) == MAX_ATTEMPTS:
113         game["status"] = "lost"
114     return feedback
```

4.3 Evidencia visual y repositorio

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9



Comodín: Wordle

Adivina la palabra española. Las tildes no cambian las letras; tienes seis intentos.

C	A	M	I	N	O
C	O	M	I	D	A
P	E	L	O	T	A
B	U	S	C	A	R

Intentos: 4/6

¡Adivinaste! La palabra era **buscar**.

Volver al castillo

Figura 3. Tablero de Wordly con cuatro intentos de seis. Se observan pistas amarillas y grises en los intentos anteriores, y la palabra BUSCAR en verde al acertarla en el cuarto intento. El botón permite volver al castillo.

Liga del código: [Abrir wordle.py de la rama Wordly en GitHub](#)

5. Conclusión

El ejercicio 19 se amplió mediante tres variantes: Pet incorpora el cuidado de una mascota y el uso de monedas; Niebla introduce una ruleta que modifica el recorrido; y Wordly agrega un reto de vocabulario con pistas por color. El código y las capturas presentados documentan cada variante en el orden Pet, Niebla y Wordly.

Para este ejercicio se creó una skill llamada Hackton, que reúne criterios para escribir Python claro y comprensible: convenciones PEP 8 y PEP 257, cuidado de la autoría, documentación e identificadores en inglés y funciones pequeñas con una responsabilidad definida.

En el repositorio consultado, la skill está registrada con el nombre hackthon y su ubicación real es `.agents/skills/hackthon/SKILL.md`. La siguiente referencia lleva directamente a ese archivo en la rama main.

Referencia de la skill: [Consultar Hackton en GitHub](#)